

خدا هست

پیش گزارش آزمایش 8 – درس آزمایشگاه DSD

امیر محمد شربتی (402106112) – پوریا رحمانی (402111418)



متأسفانه توصیف این آزمایش بسیار مختصر است و ابهامات زیادی در پیاده سازی وجود دارد. به هر حال ما تلاش کردیم مداری درست و پیچیده پیاده سازی کنیم. چیزی که استنباط ما از این آزمایش بود را شرح می دهیم:

در این آزمایش باید ALU اعداد مختلط را پیاده سازی کنیم. همچنین با اینکه هیچ اشاره ای به پیاده سازی حافظه نشده است، ولی به نظر باید یک ماژول حافظه هم پیاده سازی شود.

نکته بسیار مبهم بعدی این است که گفته شده نبایستی از واحد های جمع کننده و ضرب کننده بیش از یک بار استفاده شود. واقعا جمله مبهمی ست. ولی ما فرض کردیم که در هر کلاک فقط از یک + یا * استفاده کنیم. در واقع گویی در سطح سخت افزار محدودیت جمع کننده و ضرب کننده داریم و فقط از هر یک یکی داریم. پس ناچاریم حتی جمع را هم در چند کلاک انجام دهیم.

در این پیاده سازی 4 ماژول داریم که توضیح مبسوط هر یک در زیر ذکر شده است:

❖ ماژول جمع و تفریق کننده (complex_adder):

مطابق توصیف آزمایش یک ماژول برای این دو عمل محاسباتی پیاده سازی کردیم. طبق برداشتی که از توصیف پر از ابهام آزمایش داشتیم، باید opcode هم در ورودی می گرفتیم تا در این ماژول تعیین بشه که باید جمع انجام دهیم یا تفریق:

```
wire signed [15:0] add_out = op ? (add_a - add_b) : add_a + add_b;

always @(posedge clk, negedge resetNot) begin
    if (!resetNot) begin
        state <= 1'b0;
        done <= 1'b0;
    end else begin
        case (state)
            1'b0: begin // REAL PART
                done <= 1'b0;
                if (start) begin
                    add_a <= a_real;
                    add_b <= b_real;
                    state <= 1'b1;
                end
            end
            1'b1: begin // IMAG PART
                result[31:16] <= add_out; // This is real result (previous state)
                add_a <= a_imag;
                add_b <= b_imag;
                state <= 2'd2;
            end
            2'd2: begin
                result[15:0] <= add_out;
                done <= 1'b1;
                state <= 1'b0;
            end
        endcase
    end
end
```

توجه شود به خاطر محدودیت تعداد واحد جمع کننده باید این کار را در چند کلاک انجام دهیم.

فرض کردیم اگر $opcode = 0$ در این صورت جمع انجام شود اگر $opcode = 1$ ، تفریق انجام شود.

به نظر نکته خاص دیگری برای این ماژول نیست و همه چیز در پیاده سازی این ماژول واضح است.

❖ ماژول ضرب کننده (complex_mult):

```
IDLE: begin
    done <= 1'b0;
    if (start) begin
        state <= AC;
    end
end

AC: begin
    ac <= a_real * b_real;
    state <= BD;
end

BD: begin
    bd <= a_imag * b_imag;
    state <= AD;
end

AD: begin
    ad <= a_real * b_imag;
    state <= BC;
end

BC: begin
    result[63:32] <= ac - bd;
    bc <= a_imag * b_real;
    state <= WAIT;
end

WAIT: begin
    state <= DONE;
end

DONE: begin
    result[31:0] <= ad + bc;
    done <= 1'b1;
    state <= IDLE;
end
```

اگر اعداد مختلط ورودی را $a+bi$ و $c+di$ در نظر بگیریم، می‌دانیم ضرب این دو عدد به صورت زیر حساب میشود:

$$(a+bi)*(c+di) = (ac - bd) + (ad + bc)i$$

بنابراین باید چهار ضرب را در چهار state انجام دهیم. برای اینکه نتایج به درستی حاصل شوند و کد تمیز تر شود، چند state اضافی تر هم در نظر گرفتیم. این کد هم به درستی کار میکند.

❖ ماژول حافظه (memory):

ماژول سومی که پیاده سازی کردیم، ماژول حافظه بود. پیاده سازی این مورد هم به صورت تلویحی در توصیف آزمایش ذکر شده است.

برای پیاده سازی این ماژول به سیگنال های کنترلی برای نوشتن در حافظه و خواندن از آن نیاز داریم. همچنین افزون بر آدرس به عنوان ورودی، داده ای که آن را در حافظه می نویسیم را به عنوان ورودی و داده ای که میخوانیم را هم به عنوان خروجی می خواهیم.

در داخل این ماژول یک آرایه 32 تایی از vector های 32 بیتی در نظر گرفتیم. در واقع فرض کردیم حافظه ای 32 کلمه ای که هر کلمه 32 بیت است، در اختیار داریم. (32 کلمه در توصیف آزمایش بود ولی 32 بیتی نبود):

```
reg signed [[31:0]] mem [0:31]; // 32  
  
always @(posedge clk) begin  
    if (write_enable)  
        mem[address] <= write_data;  
    else if (read_enable)  
        read_data <= mem[address];  
end
```

این یک حافظه ساده ی سنکرون است که به نظر پیاده سازی آن خیلی واضح است...

❖ ماژول کنترلر (complex_number_ALU):

ماژول اصلی و دشوار این تکلیف همین ماژول است. باید تلاش کنیم در قالب یک پردازنده این ماژول های محاسباتی که توضیح آن ذکر شد، را مدیریت کنیم.

در پیاده سازی این مدار از state های بسیاری استفاده شده است.

متأسفانه توضیحات مربوط به این بخش هم در توصیف آزمایش ناچیز است ولی برداشت ما چنین بود:

اول اینکه فرض کردیم pc یا همان program counter در ابتدا صفر است و یکی یکی بیشتر میشود. یعنی اینکه دستورات از خانه شماره صفر اجرا میشوند. (میتوان طوری مدار را طراحی کرد که مثلاً pc را از بیرون به آن دهیم تا از خانه دلخواه حافظه دستورات را اجرا کند.)

نکته دیگر این است که فرض کردیم دستورات به این فرمت هستند:

Opcode (4 bit) + address_of_first_operand (5 bit) + address_of_second_operand (5 bit) + address_of_destination (5 bit) + 13 bit nothing

در واقع فرض شد 4 بیت اول opcode هستند. 5 بیت بعدی آدرس عملگر اول، 5 بیت بعدی آدرس عملگر دوم، 5 بیت بعدی آدرسی که باید حاصل را در آن ذخیره کنیم و مابقی 13 بیت هم بی اهمیت هستند.

نکته دیگری که فرض کردیم این است که فرض کردیم اعداد مختلط بدین صورت در حافظه ذخیره میشوند که 16 بیت سمت چپ مربوط به بخش حقیقی و 16 بیت سمت راست مربوط به بخش موهومی هستند.

نکته دیگری که فرض کردیم این است که ضرب دو عدد 32 بیتی چون 64 بیت است پس باید دو خانه از حافظه را به حاصل اختصاص دهیم. پس آدرس مقصد را به ذخیره کردن بخش موهومی و یک آدرس بیشتر را به ذخیره کردن بخش حقیقی اختصاص دادیم.

مراحل اصلی اجرای این عملیات ها همان `FETCH, DECODE, EXECUTE, WRITE_BACK` هستند. چون این کد طولانی است از قرار دادن عکس آن در گزارش خودداری میکنم.

حال که تمام ماژول های مورد نظر را پیاده سازی کردیم نوبت به تست ماژول ها می رسد.

برای ماژول تست (`complex_ALU_tb`) چند داده (عدد) و چند دستور را در حافظه ذخیره میکنیم. مطابق فرضی که قبلا هم توضیح داده شد، دستورات را از خانه شماره صفر حافظه شروع میکنیم. برای مثال در ماژول تستی که نوشتیم دو عدد $3+2i$ و $1+4i$ را به ترتیب در خانه های 16 و 17 حافظه ذخیره کردیم. سپس سه دستور را در خانه های شماره صفر و یک و دو ذخیره کردیم که به ترتیب جمع، ضرب و تفریق این دو عدد هستند:

```
// Operand A = 3 + 2i
DUT.MEM.mem[16] = {16'd3, 16'd2};

// Operand B = 1 + 4i
DUT.MEM.mem[17] = {16'd1, 16'd4};

// Instruction: ADD: mem[16] + mem[17] -> mem[3]
DUT.MEM.mem[0] = {4'b0000, 5'd16, 5'd17, 5'd3, 13'd0};

// Instruction: MUL: mem[16] * mem[17] -> mem[4]
DUT.MEM.mem[1] = {4'b0010, 5'd16, 5'd17, 5'd4, 13'd0};

// Instruction: SUB: mem[16] - mem[17] -> mem[6]
DUT.MEM.mem[2] = {4'b0001, 5'd16, 5'd17, 5'd6, 13'd0};
```

معنی تمام دستورات هم در کد کامنت شده است.

پس از اینکه `done = 1` شود، نتایج را بررسی میکنیم. نتیجه پرینت شده در کنسول برای این سه دستورا به صورت زیر است که کاملا مطابق انتظار است:

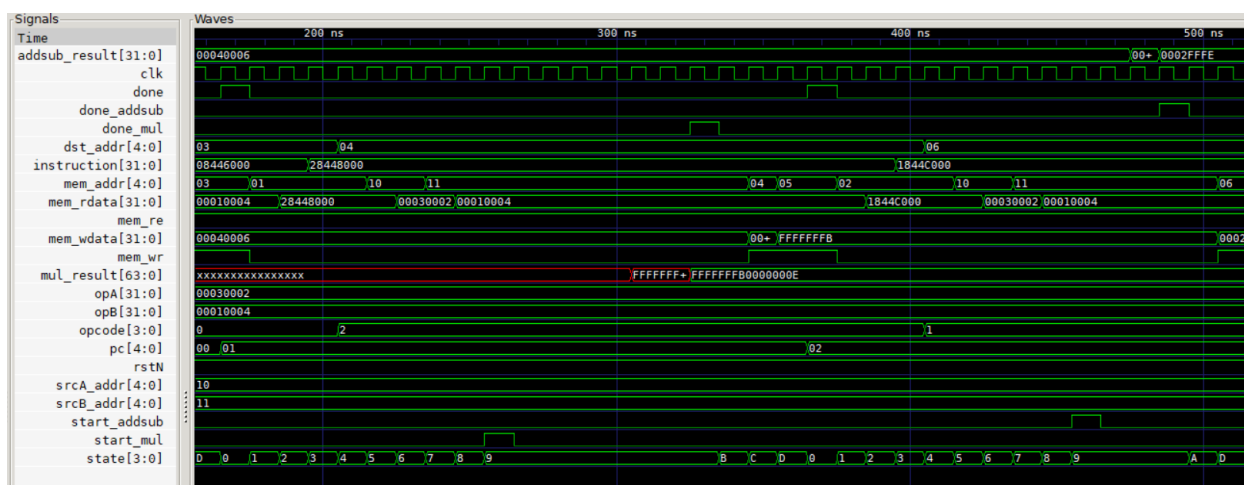
```

C:\Users\ASUS\OneDrive\Desktop\university\3\DSB_El
VCD info: dumpfile complex_ALU.vcd opened for output.
ADD result = 4 + 6i
MUL result = -5 + 14i
SUB result = 2 + -2i
.\complex_ALU_tb.v:66: $finish called at 515000 (1ps)

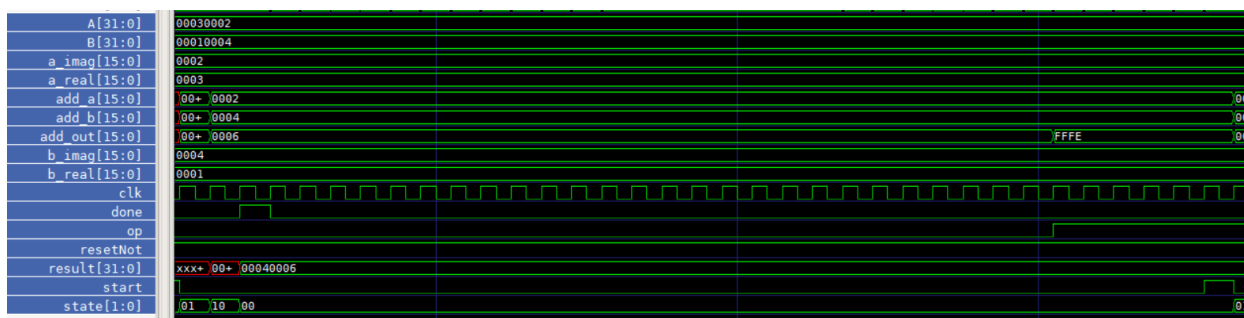
```

همانطور که مشخص است همه چیز مطابق انتظار است.

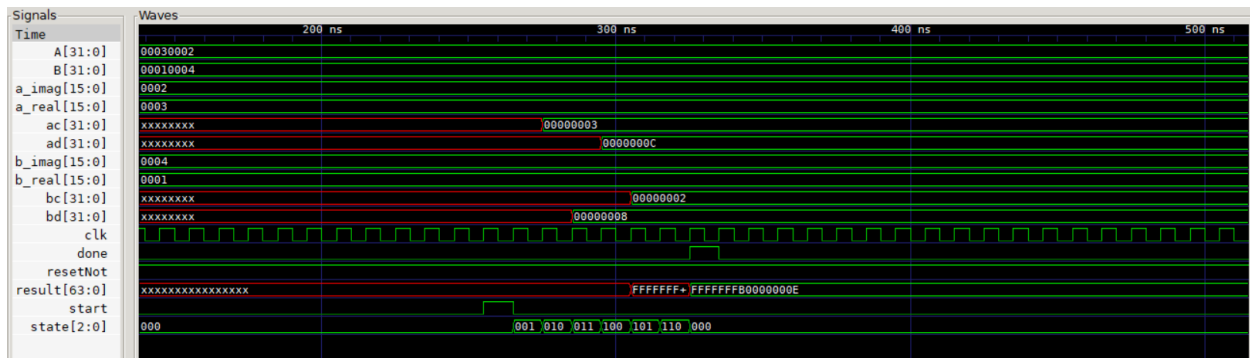
حال waveform همین مازول تست را به کمک GTK مشاهده میکنیم. این waveform بسیار پیچیده است ولی بعد چند عکس از آن را در اینجا قرار میدهم. (این نمودار خیلی خیلی در دیباگ کد به ما کمک کرد.)



این waveform سیگنال های مازول اصلی (complex_number_ALU) است.



این هم برای مازول جمع/تفریق است.



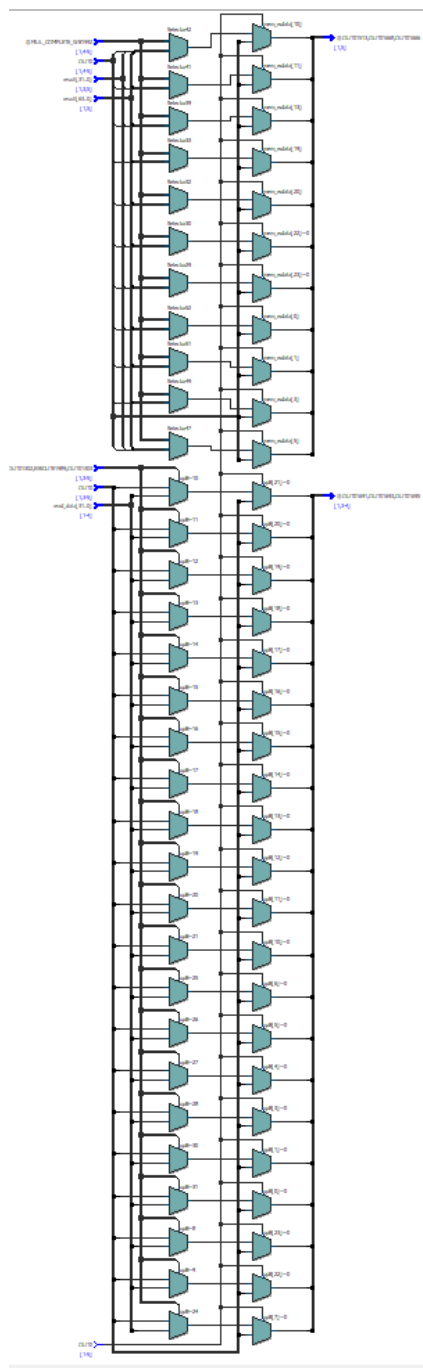
این هم مربوط به سیگنال های ماژول ضرب کننده است.

پس این کد از نظر منطقی کاملاً درست کار میکند.

نکته دیگری که باید در گزارش ذکر شود، سنتز پذیری کد است. این کد را در کوارتوس اجرا کردیم و خدا رو شکر بدون مشکل سنتز شد. این هم نتایج flow summary و RTL viewer است:

The screenshot shows the Quartus II 64-bit Web Edition interface. The top menu bar includes File, Edit, View, Project, Assignments, Processing, Tools, Window, and Help. The Project Navigator on the left shows the project files: complex_mult.v, complex_adder.v, complex_number_ALU.v, memory.v, and complex_ALU_tb.v. The main window displays the 'Flow Summary' for the 'complex_number_ALU' project. The flow status is 'Successful - Sun Dec 28 22:02:49 2025'. The summary includes details about the Quartus II version (13.1.0), revision name (complex_number_ALU), top-level entity name (complex_number_ALU), family (Cyclone IV GX), and resource usage statistics. The Messages window at the bottom shows various warnings and successful completion messages, including the command used to generate the netlist: 'quartus_eda --read_settings_files=off --write_settings_files=off complex_number_ALU_6_1200mv_85c_slow.vo in folder "C:/Users/ASUS/OneDrive/Desktop/ur'.

عکس زیر مربوط به RTL مدار اصلی (complex_number_ALU) است:



فایل تمام این نتایج هم به پیوست ارسال شدند.

خدا رو شکر همه چیز مطابق انتظار است و همانطور که توضیح داده شد، کد به درستی کار میکند و همچنین سنتز پذیر است.